Bảy kịch bản cải tiến truy hồi cho hệ hỏi đáp

Thành phần công nghệ, thuật toán, triển khai, kết quả thực đo và khuyến nghị lựa chọn. Mỗi kịch bản là một gói mã nguồn độc lập, chạy được, kèm ablation tự kiểm chứng.

Tổng quan — Kiến trúc tích luỹ 1

S1 · Hybrid — dense + BM25 + RRF 3

S2 · Chunking — Giữ ngữ cảnh 4

S3 · Rerank — Cross-encoder 5

S4 · Phạm vi — Lọc phiên bản 6

S5 · Truy vấn — Query understanding 7

S6 · Ngữ cảnh — adaptive-k, MMR 8

S7 · Quyết định — Hiệu chuẩn, từ chối 9

So sánh — Bảng tổng hợp 11

Phát hiện — Xuyên suốt 7 kịch bản 12

Khuyến nghị — Nên dùng gì 13

Lộ trình — Thứ tự triển khai 14

00

Tổng quan

Bảy kịch bản được thiết kế **tích luỹ**: mỗi gói kế thừa toàn bộ mã nguồn của kịch bản trước và chỉ thêm đúng một biến. Nhờ vậy khi một chỉ số thay đổi, nguyên nhân quy được về một thành phần duy nhất.

```
truy vấn
  → S5  query understanding     chuẩn hoá, fold dấu, từ điển, tách câu
  → S4  lọc phạm vi             commit/phiên bản/phân khúc — TRƯỚC khi chấm điểm
  → S1  dense + BM25 → RRF      hai nhánh song song, hợp nhất theo thứ hạng
  → S3  cross-encoder rerank    chấm lại top-N, sinh điểm hiệu chuẩn được
  → S7  quyết định              answer | conditional | evidence_only | abstain
  → S6  chọn ngữ cảnh           adaptive-k, MMR, thứ tự chữ U
        ↑ S2 chunking tác động lên toàn bộ qua embed_text
```

Mỗi gói gồm: mã nguồn đầy đủ (kể cả phần kế thừa không đổi), một runner chạy ablation, một corpus demo chạy được không cần GPU, và README. Tổng 2.786 dòng mã ở gói S7 — gói đầy đủ nhất.

Nguyên tắc xuyên suốt. Mọi runner đều có *điều kiện dừng* — nếu kết quả không đạt, script trả mã lỗi khác 0 và in lý do. Mục tiêu không phải tạo ra bảy cải tiến, mà là biết kịch bản nào thực sự giúp được trên dữ liệu của bạn và kịch bản nào không.

01

Hybrid retrieval

S1 — dense + BM25 + RRF

BẮT BUỘC

Thành phần

Qwen3-Embedding (bi-encoder) · BM25 tự triển khai trên scipy sparse · Reciprocal Rank Fusion (k=60)

Thuật toán

Cosine trên vector chuẩn hoá; BM25 với ma trận trọng số tiền tính; hợp nhất `Σ w/(k+rank)`

Mã nguồn

`embedding.py` · `retriever.py` · `run_s1.py`

Chỉ số

recall@k, nDCG@10, so ba chế độ `dense | bm25 | hybrid`

Chế độ lỗi

Retrieval thất bại — thông tin có trong corpus nhưng không lọt top-k

Vì sao RRF chứ không trộn điểm. BM25 cho điểm dương không chặn, cosine bị chặn trong [-1,1]. Trộn điểm thô mặc nhiên cho BM25 trọng số áp đảo. RRF chỉ thao tác trên thứ hạng nên không cần giả định về thang điểm.

ĐIỂM MẠNH

- Chi phí thấp nhất trên mỗi điểm recall thu được
- Không cần huấn luyện, không tham số học
- Dễ debug — tách được lỗi thuộc nhánh nào
- BM25 bền với thuật ngữ nội bộ, mã lỗi, tên gói cước mà embedding chưa từng thấy

ĐIỂM YẾU

- Vẫn xếp hạng từng chunk độc lập, không nối chúng lại
- Duy trì hai index, tăng chi phí vận hành
- **Tăng mật độ distractor** trong tập ứng viên — giúp recall nhưng không giúp hallucination
- Trọng số hợp nhất phải tinh chỉnh theo miền

Điều kiện dừng: hybrid phải thắng cả hai nhánh đơn. Nếu không, vấn đề ở chunking hoặc ở tập eval, không phải ở model — và không nên sang S2.

02

Chunking giữ ngữ cảnh

S2 — structure-aware + khối nguyên tử + enrichment

BẮT BUỘC

Thành phần

3 chiến lược: `fixed` (baseline) · `structure` (heading/đoạn) · `ast` (cây cú pháp Python) · bảo vệ khối nguyên tử · enrichment header

Thuật toán

cAST — chia đệ quy theo node AST, gộp node anh em trong giới hạn kích thước; nhận diện span nguyên tử (code fence, bảng, list, traceback) rồi đóng gói không chia cắt

Mã nguồn

`chunking.py` · `run_s2.py`

Chỉ số

`fragmentation_rate`, `answer_recall@k`, `span_coverage@k`

Chế độ lỗi

Hiểu sai ngữ cảnh — model ghép nửa dữ kiện này với nửa dữ kiện kia

Kết quả thực đo: fixed-700 làm mất **21,4%** số span (3/14 không chunk nào chứa trọn); structure đưa về 0 và nâng `span_coverage@5` từ 0.722 lên 0.889, đổi lại 1,71 lần số chunk.

> **Bẫy phương pháp.** `gold_ids` của S1 vô dụng ở đây — đổi chiến lược chunking là đổi toàn bộ tập chunk, mọi id đều khác. S2 phải đánh giá theo **span trong tài liệu gốc**. Dùng lại nhãn cũ sẽ cho một bảng số trông bình thường nhưng đo một thứ vô nghĩa.

`fragmentation_rate` là chỉ số đáng theo dõi nhất trong cả bảy kịch bản: nó đếm số span mà **không chunk nào chứa trọn** — những câu trả lời không thể cứu dù retrieval hoàn hảo. Và nó tính được mà không cần chạy retrieval lần nào.

ĐIỂM MẠNH

- Tác động lên *mọi* tầng phía sau — cả hai nhánh đều đọc cùng `embed_text`
- Mức cải thiện lớn nhất trên mỗi giờ công
- Chẩn đoán rẻ nhất: đo được trước khi chạy retrieval
- Không cần GPU, không cần LLM

ĐIỂM YẾU

- Phải re-index toàn bộ mỗi lần thay đổi
- Chi phí lưu trữ tăng (demo: 1,71×)
- AST chunker chỉ dùng được cho mã nguồn, cần parser riêng mỗi ngôn ngữ
- Không có kích thước chunk tối ưu phổ quát — vẫn phải quét

03

Cross-encoder reranking

S3 — rerank như một thí nghiệm có thể bác bỏ

CÓ ĐIỀU KIỆN

Thành phần
bge-reranker-v2-m3 (seq-cls) hoặc Qwen3-Reranker (yes/no logit) · hai chế độ hợp nhất `replace` / `rrf`

Thuật toán
Chấm điểm cặp (query, doc) qua cross-encoder, sigmoid về [0,1]; chế độ `rrf` trộn thứ hạng CE với thứ hạng tầng một

Mã nguồn
`rerank.py` · `retriever.py` (sửa) · `run_s3.py`

Chỉ số
precision@k (không chỉ recall), latency p50/p95, `rank_churn`, `overlap` (sẵn sàng cho S7)

Vì sao có điều kiện. Cross-encoder huấn luyện trên MS MARCO đã được đo **làm giảm** precision ở mọi mức cutoff trên corpus ngoài miền — P@5 rơi từ 0.600 xuống 0.404. Reranker cần được kiểm chứng trên chính miền dữ liệu trước khi triển khai. Trong demo, chế độ `replace` làm tụt cả precision lẫn recall, còn `rrf-blend` kéo về bằng baseline.

> **Đóng góp thật của S3 không nằm ở sắp xếp lại.** Nó nằm ở chỗ tạo ra một **đại lượng so sánh được giữa các truy vấn** để S6 cắt adaptive-k và S7 đặt ngưỡng từ chối. Điểm RRF phẳng và chỉ có nghĩa trong phạm vi một truy vấn. Nên ngay cả khi S3 bị bác bỏ về precision, vẫn giữ reranker làm nguồn điểm.

ĐIỂM MẠNH

- Khi hợp, cho mức cải thiện precision lớn nhất trong một bước
- Tách rời khỏi embedding — đổi reranker không cần re-index
- Sinh ra điểm hiệu chuẩn được, điều kiện cần của S6 và S7
- Chế độ `rrf` chặn được việc CE ghi đè tầng một đã đúng

ĐIỂM YẾU

- **Có thể làm hại** trên corpus ngoài miền huấn luyện
- Chi phí tuyến tính theo số ứng viên; ảnh hưởng latency rõ rệt
- Không sửa được lỗi recall — tài liệu không lọt ứng viên thì rerank vô nghĩa
- Không phát hiện được chunk sai phiên bản (xem S4)
- Nhánh Qwen3 chưa kiểm chứng định dạng prompt

04

Lọc phạm vi và phiên bản

S4 — pre-filter theo metadata

NÊN CÓ

Thành phần

Schema metadata (status, effective_from/to, audience, doc_type, version) · trình biên dịch bộ lọc · trích xuất hiệu lực từ văn bản · ba chế độ `pre | post | off`

Thuật toán

Mask boolean trên index, áp vào cả hai nhánh *trước* khi chấm điểm; không khớp gì thì trả rỗng kèm lý do

Mã nguồn

`metadata.py` · `retriever.py` (sửa) · `run_s4.py`

Chỉ số

`stale_rate@k`, `stale_outranks_current`, độ phủ metadata

Kết quả thực đo, và cả ba đều đáng lưu:

Biến thể	rec@5	stale@5	cũ xếp trên mới	trả về
không lọc	1.000	1.000	0.571	5.0
không lọc + rerank	1.000	1.000	0.571	5.0
post-filter	1.000	0.000	0.000	**2.0**
pre-filter	1.000	0.000	0.000	5.0

- **Recall hoàn toàn mù.** 1.000 ở mọi biến thể, kể cả khi 100% truy vấn có chunk hết hiệu lực trong ngữ cảnh.
- **Rerank không cứu được.** Cross-encoder không có cách nào biết một chunk đã hết hiệu lực — thông tin đó không nằm trong văn bản.
- **Post-filter âm thầm phá top-k.** Trả 2 chunk thay vì 5, không có gì báo điều đó.

ĐIỂM MẠNH

- Cơ chế duy nhất xử lý được hard negative "đúng chủ đề sai phiên bản"
- Tất định, không dựa trên xác suất — đáng tin hơn mọi guardrail dựa trên điểm
- Chi phí tính toán gần bằng 0
- Làm sạch tập ứng viên nên **nâng giá trị của S3**

ĐIỂM YẾU

- Đòi metadata phủ 100% — nếu không có quy trình gắn, phải tự suy ra và suy sai thì lọc mất câu đúng
- **Không phát hiện được câu hỏi thuộc phạm vi khác** — chỉ giới hạn tài liệu đủ điều kiện
- Hỏng use case tra cứu lịch sử nếu lọc cứng `status=current`
- Vô dụng nếu corpus vốn không có nội dung chồng phiên bản

05

Query understanding

S5 — chuẩn hoá, fold dấu, từ điển, tách câu

NÊN CÓ

Thành phần

Chuẩn hoá NFC · **fold dấu tiếng Việt** · từ điển viết tắt/thông tục · phân loại truy vấn bằng luật · viết lại và tách câu (rule hoặc LLM)

Thuật toán

Sinh nhiều biến thể truy vấn, retrieve song song, hợp nhất RRF với trọng số giảm dần `decay**i`; trọng số dense/BM25 chọn theo lớp truy vấn

Mã nguồn

`query_understanding.py` · `retriever.py` (sửa tokenizer + hợp nhất đa biến thể) · `run_s5.py`

Chỉ số

recall@1 **theo nhóm**: plain / noaccent / abbrev / multipart; số truy vấn bị từng cơ chế làm hỏng

Fold dấu là thay đổi phía index, không phải tham số truy vấn. Index BM25 phải lưu cả `mật` lẫn `mat`. Nếu giữ index cố định rồi chỉ bật/tắt phía truy vấn thì cơ chế này trông vô dụng — index đã làm sẵn phần việc đó.

Kết quả: fold dấu nâng nhóm `noaccent` +0.20 và `abbrev` +0.50, nhưng **làm hỏng** một truy vấn multipart (−0.33). Chấm gộp sẽ không thấy gì trong số này.

> **Quy tắc an toàn.** Truy vấn gốc luôn là biến thể số 0 với trọng số cao nhất và không bao giờ bị thay thế. Một bản viết lại lệch hướng vì thế chỉ làm xê dịch thứ hạng chứ không làm mất câu trả lời.

ĐIỂM MẠNH

- Fold dấu giải quyết một vấn đề rất thật của tiếng Việt mà hệ quốc tế không gặp
- Từ điển rẻ, dễ bảo trì, hiệu quả ngay trên nhóm viết tắt
- Phân loại bằng luật nên kiểm toán được
- Không cần LLM ở cấu hình mặc định

ĐIỂM YẾU

- Mỗi biến thể là một lần gọi embedding và một lượt BM25 — latency nhân lên
- Fold dấu **có thể làm hỏng** một số truy vấn (đo được trong demo)
- Từ điển phải viết tay và cập nhật theo miền
- Viết lại bằng LLM có rủi ro lệch hướng, nên tắt mặc định
- Vô dụng nếu tập eval chỉ có câu hỏi chuẩn

06

Chọn ngữ cảnh

S6 — adaptive-k (nhận) · MMR và U-shape (có điều kiện)

NÊN CÓ

Thành phần
adaptive-k theo độ dốc điểm + ngưỡng sàn · MMR · thứ tự chữ U

Thuật toán
Cắt tại vị trí đầu tiên có `score[i] < gap_ratio × score[i−1]`; MMR greedy `λ·sim(q,d) − (1−λ)·max sim(d, đã chọn)`; sắp lại `[1,2,3,4,5] → [1,3,5,4,2]`

Mã nguồn
`context_selection.py` · `retriever.py` (sửa) · `run_s6.py`

Chỉ số
`avg_distractors`, `avg_k`, `redundancy`, `edge_dist`, `cut_reason`

Đại lượng cần tối thiểu hoá không phải "nội dung không liên quan" nói chung, mà số chunk *đúng chủ đề nhưng không chứa đáp án*. Tài liệu liên quan nhưng không chứa đáp án gây hại hơn hẳn tài liệu hoàn toàn không liên quan: chỉ một tài liệu gây nhiễu đã làm giảm độ chính xác tới 25%, tăng dần vượt 67% khi thêm nhiều — trong khi thêm tài liệu *ngẫu nhiên* lại cải thiện tới 35%.

Kết quả: adaptive-k bỏ **1,70 distractor mỗi truy vấn với recall không đổi**. Sweep `gap_ratio` cho cả đường cong, từ 5,0 distractor (k cố định) xuống 0,8 (đổi lại mất 10 điểm recall).

> **Hai tương tác được phát hiện tự động.** MMR cho `redundancy_drop` = 0 — ngữ cảnh vốn không trùng lặp, vì chunking đã gộp sẵn các đoạn gần giống nhau; nên tắt. Và **adaptive-k với U-shape triệt tiêu nhau**: reordering chỉ có giá trị trên tập truy hồi lớn, mà adaptive-k tồn tại để làm tập nhỏ. Demo cho U-shape chỉ kích hoạt 50% truy vấn.

ĐIỂM MẠNH

- adaptive-k: chi phí bằng 0, tác động trực tiếp lên hallucination
- Không cần re-index, quét được cả đường cong trong vài phút
- `cut_reason` cho biết ngay khi cơ chế không hoạt động
- Đo được thứ mà recall@k hoàn toàn mù

ĐIỂM YẾU

- **Phụ thuộc S3** — trên điểm RRF phẳng thì phép cắt kích hoạt gần như ngẫu nhiên
- MMR thường trơ nếu chunking đã khử trùng lặp
- U-shape xung đột với adaptive-k, và hiệu quả chỉ đo được ở tầng generation
- Cắt quá tay thì mất recall — phải quét, không có giá trị mặc định đúng

07

Hiệu chuẩn và quyết định trả lời

S7 — thang bốn mức, ngưỡng suy từ chi phí

BẮT BUỘC

Thành phần
Hiệu chuẩn hai phân phối điểm · đường cong đánh đổi · chọn ngưỡng theo `cost_ratio` · thang 4 mức · tín hiệu margin

Thuật toán
`cost = cost_ratio × false_answer_rate + over_abstention`, chọn ngưỡng cực tiểu; ba ranh giới suy ra từ phân vị của hai phân phối; margin `top1−top2` dưới ngưỡng thì hạ một bậc

Mã nguồn
`confidence.py` · `retriever.py` (sửa) · `run_s7.py`

Chỉ số
`false_answer_rate`, `over_refusal`, `coverage`, `separation`, phân rã theo loại câu không trả lời được

Kết quả: `false_answer_rate` từ **1.000 xuống 0.091**, đổi lại coverage 1.000 → 0.667.

Bốn mức thay vì từ chối nhị phân

Mức	Nghĩa	may_assert
`answer`	Bằng chứng mạnh, trả lời bình thường	✓
`answer_conditional`	Dùng được, nêu kèm điều kiện và trích dẫn	✓
`evidence_only`	Chưa đủ kết luận, hiển thị đoạn liên quan nhất	✗
`abstain`	Không có gì đáng hiển thị	✗

`evidence_only` là mức quan trọng nhất cho hệ phục vụ người dùng cuối: chi phí hallucination bằng 0 vì không khẳng định gì, mà người dùng vẫn nhận được thứ để dùng.

> **`cost_ratio` là tham số bạn phải tự chốt.** Chi phí một câu trả lời sai tự tin chia cho chi phí một lần từ chối không cần thiết. Ở mức 20, hệ thống đạt `false_answer_rate` = 0 nhưng coverage tụt còn 0.083 — đúng về toán học, vô dụng về sản phẩm. Chọn có chủ đích và ghi vào báo cáo, vì mọi ngưỡng phía sau kế thừa nó.

Ba loại câu không trả lời được, và loại khó nhất: `absent` (chủ đề không có) 0.00 bị khẳng định · `adjacent` (chủ đề có, dữ kiện không) **0.20** · `oos` (ngoài phạm vi) 0.00. Khoảng trống mà S4 để lại đã được S7 bịt.

ĐIỂM MẠNH

- Cơ chế duy nhất trong tầng retrieval chặn được câu trả lời sai tự tin
- Thang bốn mức giữ được tính hữu dụng, không đánh đổi tất-cả-hoặc-không
- Ngưỡng suy từ dữ liệu và một tham số chi phí tường minh
- Bịt được khoảng trống phạm vi mà S4 không xử lý
- `may_assert` là hợp đồng rõ ràng với tầng generation

ĐIỂM YẾU

- **Phụ thuộc hoàn toàn vào S3** — `separation` dưới 0.3 thì không ngưỡng nào cứu được
- Đòi tập eval có 20–30% câu không trả lời được, phải xây tay
- Mất coverage là chi phí thật, không tránh được
- Vô hiệu nếu tầng generation không tôn trọng `may_assert`
- Không kiểm tra câu trả lời có bám vào bằng chứng hay không — đó là việc của tầng sau

08

Bảng so sánh tổng hợp

KB	Chế độ lỗi nhắm tới	Chi phí index	Chi phí truy vấn	Re-index?	Phụ thuộc	Rủi ro chính
S1	Retrieval thất bại	Trung bình	Thấp	Có	—	Tăng mật độ distractor
S2	Hiểu sai ngữ cảnh	Thấp	Không	Có	—	Tăng số chunk, phải quét kích thước
S3	Retrieval thất bại + sinh điểm	Không	Cao	Không	S1	Có thể giảm precision ngoài miền
S4	Tài liệu sai phiên bản	Thấp	Gần 0	Metadata	Metadata	Thiếu metadata thì lọc rò hoặc lọc nhầm
S5	Lỗi phía đầu vào	Thấp	Trung bình	Có (fold)	S1	Latency nhân theo số biến thể
S6	**Tài liệu gây nhiễu**	Không	Gần 0	Không	**S3**	Cắt quá tay thì mất recall
S7	Khẳng định không có cơ sở	Không	Gần 0	Không	**S3**	Mất coverage; vô hiệu nếu generation bỏ qua

Tác động lên hallucination so với tác động lên recall

Hai trục này **không trùng nhau**, và đây là kết luận quan trọng nhất của toàn bộ công việc.

Kịch bản	Tác động lên recall	Tác động lên hallucination
S1 Hybrid	Cao	Thấp — còn làm tăng distractor
S2 Chunking	Cao	Cao
S3 Rerank	Trung bình (có thể âm)	Gián tiếp — qua điểm hiệu chuẩn
S4 Lọc phạm vi	**Không** (recall mù)	**Rất cao**
S5 Query	Cao ở nhóm đầu vào xấu	Trung bình
S6 adaptive-k	**Không**	**Cao**
S7 Quyết định	Âm (giảm coverage)	**Rất cao**

Ba kịch bản có tác động cao nhất lên hallucination — S4, S6, S7 — đều **không cải thiện recall**, và hai trong số đó làm recall hoặc coverage giảm. Nếu chỉ đo recall, bạn sẽ loại đúng ba thứ cần giữ nhất.

09

Phát hiện xuyên suốt

1. Lỗi tokenizer làm hỏng BM25 trên toàn bộ tiếng Việt

Tokenizer kế thừa từ baseline dùng regex `[^0-9A-Za-z_]+`, viết cho định danh code ASCII. Trên tiếng Việt nó coi mọi ký tự có dấu là ký tự ngăn cách:

```
tokenize("mật khẩu")  ->  ['m', 't', 'kh', 'u']        # trước
tokenize("mật khẩu")  ->  ['mật', 'mat', 'khẩu', 'khau']  # sau
```

Không crash, không cảnh báo — nhánh lexical chỉ đơn giản gần như không đóng góp gì. Đã sửa thành `\W+` với `re.UNICODE` và backport sang cả bảy gói. **Bài học:** baseline kế thừa từ miền khác mang theo giả định của miền đó; chạy tokenizer trên vài câu tiếng Việt thật trước khi tin bất kỳ con số BM25 nào.

2. Recall@k mù trước hai lớp lỗi nguy hiểm nhất

Ở S4, recall@5 bằng 1.000 ở mọi biến thể trong khi 100% truy vấn có chunk hết hiệu lực và 57% truy vấn bản cũ xếp trên bản mới. Ở S6, cả bốn biến thể đều recall 1.000 trong khi số distractor đi từ 5,0 xuống 3,3. **Chỉ đo recall là cách chắc chắn nhất để bỏ lỡ cả hai.**

3. Nhãn eval không bền qua các kịch bản

`gold_chunk_id` dùng được ở S1 nhưng chết ngay khi S2 đổi chunker. Tập eval phải chuyển sang **span trong tài liệu gốc** từ S2 trở đi, và bổ sung nhãn phiên bản ở S4, nhãn nhóm truy vấn ở S5, nhãn không-trả-lời-được ở S7. Xây tập eval theo cách này ngay từ đầu rẻ hơn nhiều so với làm lại bốn lần.

4. Hai cặp cơ chế triệt tiêu nhau

- **MMR ↔ khử trùng lặp ở chunking.** Nếu S2 đã gộp các đoạn gần giống nhau, MMR không còn gì để làm.
- **adaptive-k ↔ U-shaped ordering.** Reordering chỉ có giá trị trên tập lớn; adaptive-k tồn tại để làm tập nhỏ. Phải chọn một.

5. Một thành phần có thể bị bác bỏ ở vai trò này mà vẫn cần cho vai trò khác

S3 có thể không cải thiện precision, nhưng vẫn là nguồn điểm duy nhất đủ tin cậy cho S6 và S7. "Bác bỏ rerank" không có nghĩa là xoá reranker khỏi pipeline.

10

Khuyến nghị: nên dùng gì

Bắt buộc — ba kịch bản không thể bỏ

KB	Lý do	Điều kiện chấp nhận
S1 Hybrid	Không có bằng chứng đúng thì mọi thứ phía sau vô nghĩa	Hybrid thắng cả hai nhánh đơn
S2 Chunking	Chunk cắt hỏng là câu trả lời không thể cứu dù retrieval hoàn hảo	`fragmentation_rate` = 0
S7 Quyết định	Cơ chế duy nhất chặn câu trả lời sai tự tin	`separation` ≥ 0.3

Nên có — tuỳ đặc điểm corpus và người dùng

- **S6 chỉ riêng adaptive-k.** Chi phí bằng 0, bỏ được 1,7 distractor mỗi truy vấn mà không mất recall. **Bỏ MMR và U-shape** trừ khi đo được lợi ích cụ thể — trong demo cả hai đều trơ.
- **S4** nếu corpus có nội dung chồng phiên bản. Kiểm tra trong một buổi: lấy 20–30 câu hỏi thật, đếm tỷ lệ có nhiều hơn một đoạn trả lời được mà mâu thuẫn nhau. Trên 10% thì làm đầy đủ; dưới 10% thì chỉ cần một trường `status` và một dòng lọc; 0% thì bỏ.
- **S5 chỉ riêng fold dấu và từ điển.** Với người dùng Việt gõ trên điện thoại, đây là hai cơ chế rẻ và hiệu quả. **Tắt viết lại bằng LLM** ở giai đoạn đầu.

Có điều kiện — phải đo trước khi quyết

- **S3 rerank.** Chạy như thí nghiệm có thể bác bỏ, đo `precision@5` chứ không chỉ recall. Ba kết luận có thể: nhận; thử `fusion="rrf"` rồi nhận; hoặc **giữ chỉ để lấy điểm cho S6 và S7**, không dùng thứ tự. Kết luận thứ ba vẫn là lý do đủ để giữ reranker trong pipeline.

Cấu hình đề xuất cho POC

```
S2  structure chunking + enrichment header + bảo vệ khối nguyên tử
S1  Qwen3-Embedding-0.6B + BM25 (tokenizer Unicode + fold dấu) + RRF k=60
S5  chuẩn hoá + fold dấu + từ điển  (tắt rewrite LLM)
S4  pre-filter status=current       (nếu corpus có chồng phiên bản)
S3  bge-reranker-v2-m3, candidates=50, fusion đo cả hai chế độ
S6  adaptive-k gap_ratio≈0.55       (tắt MMR, tắt U-shape)
S7  thang 4 mức, cost_ratio chốt theo nghiệp vụ
```

Bảy kịch bản nhưng cấu hình thực tế chỉ bật khoảng mười cơ chế. Phần còn lại là những thứ đã được đo và loại bỏ có bằng chứng — và đó cũng là kết quả, không phải lãng phí.

11

Lộ trình triển khai

Đợt	Việc	Thời gian	Kết quả cần có
0	Xây tập eval: 100–200 câu, **span trong tài liệu gốc**, 20–30% không trả lời được, gắn nhãn nhóm truy vấn và phiên bản	2–3 ngày	Mọi kịch bản sau đều đo được
1	S1 + S2 — nền tảng và chunking	3 ngày	Hybrid thắng hai nhánh đơn; `fragmentation_rate` = 0
2	S4 (nếu cần) + S3 — chốt cùng nhau	2 ngày	`stale_rate` = 0; kết luận rõ về rerank
3	S6 adaptive-k + S7 hiệu chuẩn	2 ngày	Đường cong đánh đổi, `cost_ratio` đã chốt
4	S5 — sau khi có log truy vấn thật	2 ngày	Phân bố 4 nhóm truy vấn từ dữ liệu thật

Đợt 0 không bỏ qua được. Không có tập eval thì bảy kịch bản chỉ là bảy cấu hình, không có cách nào biết cái nào tốt hơn. Và tập eval phải xây đúng từ đầu: nhãn span, nhóm câu không trả lời được, nhãn phân nhóm truy vấn. Làm lại bốn lần tốn hơn nhiều so với làm đúng một lần.

Đợt 4 đặt cuối có lý do: S5 tối ưu cho dạng đầu vào thật, mà dạng đó chỉ biết được sau khi hệ thống chạy và có log. Làm sớm thì tối ưu cho giả định.

Hai việc nằm ngoài tầng này nhưng cần phối hợp

- **Tầng generation phải tôn trọng `may_assert`.** Nếu nhận 0 chunk rồi vẫn trả lời bằng tri thức nội tại của model thì toàn bộ S7 vô nghĩa. Đây là điểm dễ vỡ nhất khi ghép tầng.
- **Kiểm chứng grounding đầu ra.** Bảy kịch bản đảm bảo *bằng chứng đúng và đủ tin cậy*, nhưng không đảm bảo *câu trả lời bám vào bằng chứng*. Model vẫn có thể nhận 3 chunk đúng rồi suy diễn thêm. Đó là việc của tầng sau.

Báo cáo mô tả trạng thái tại 07/10/2026. Mọi con số kết quả lấy từ ablation trên corpus demo với embedder và reranker giả lập — chúng minh hoạ cơ chế và dạng lỗi, không phải mức chất lượng tuyệt đối. Phải đo lại trên dữ liệu và model thật trước khi dùng làm cơ sở quyết định.